

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 4
Operating Systems
Topic: The xv6 Shell from the Outside

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: FIX SLEEP FOR THE CURRENT XV6 **4**

2.1 System Call Interface Verification 4

2.2 Source Code Implementation 4

2.3 Execution and Verification 4

3 PART 2: BASH VERSUS THE XV6 SHELL **5**

3.1 Comparative Experiment Matrix 5

3.2 Real Execution Outputs 6

3.3 Analysis and Observations 7

4 PART 3: WHO EXPANDS *? ARGV DIAGNOSTICS **8**

4.1 Implementation of Diagnostic Programs 8

4.2 Experimental Output Comparison 9

4.3 Analysis and Observations 9

5 PART 4: SOURCE CODE ANALYSIS OF USER/SH.C **11**

5.1 Inspecting Command Representations 11

5.2 Analysis and Observations 12

6 CONCLUSIONS **13**

1 INTRODUCTION AND GOAL

1.1 Introduction

Command line interpreters (shells) serve as the primary interactive user interface to an operating system. While end users frequently perceive the shell and the kernel as a unified entity, UNIX architecture enforces a fundamental separation of responsibilities: the kernel provides process management, virtual memory, descriptor tables, and filesystem storage primitives, whereas the shell acts as a userland program tasked with lexical analysis, line parsing, environment configuration, command dispatching, and stream orchestration via pipes and redirections.

1.2 Lab Objective

The objective of this laboratory work is to investigate the internal architecture and operational behavior of Unix command interpreters by systematically comparing the minimal educational shell in xv6 (`user/sh.c`) with the production GNU Bash interpreter in Linux. The key study areas include:

- updating user utilities in xv6 to match the current kernel system call interface (`pause()` replacing `sleep()`);
- executing a structured matrix of shell operations in both Linux Bash and xv6 to observe features such as redirections, anonymous pipes, command grouping, sequencing, background jobs, globbing, variable substitution, and quote processing;
- evaluating argument parsing behavior by writing custom diagnostic tools (`args.c` and `args_linux.c`) to reveal who performs wildcard expansion and quote stripping;
- reading and analyzing the source code of `user/sh.c` to understand command representation and the architectural rationale for built-in commands like `cd`.

2 PART 1: FIX SLEEP FOR THE CURRENT XV6

2.1 System Call Interface Verification

In the modern revision of the xv6-riscv operating system, the system call responsible for relinquishing CPU ticks was updated to `pause()` to eliminate semantic ambiguities. The declaration was inspected in `user/user.h`:

```
vitalie@Lenovo:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

```
// user/user.h - line 25
int pause(int);
```

Figure 2.1: Figure 2.1 – System call declaration for pause() in user/user.h

2.2 Source Code Implementation

The user utility `user/sleep.c` was implemented to consume a tick duration argument and invoke the updated system call:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Figure 2.2: Figure 2.2 – The updated sleep.c implementation using pause()

The `Makefile` was verified to confirm that `$U/_sleep` is compiled into the root file system image (`fs.img`) under `UPROGS`.

2.3 Execution and Verification

The utility was compiled and tested under the xv6 QEMU emulator:

```
$ sleep
usage: sleep <ticks>
$ sleep 20
$
```

Figure 2.3: Figure 2.3 – Verification of sleep utility in xv6 with error validation and valid tick delay

3 PART 2: BASH VERSUS THE XV6 SHELL

3.1 Comparative Experiment Matrix

Ten standardized command constructions were executed in both the Linux Bash environment (within a test directory containing `README`) and the xv6 interactive shell. Table 3.1 summarizes the observed behavioral responses.

Command	Construct	Linux Bash	xv6 Shell
`echo hello > f; cat < f`	I/O Redirection	Prints `hello`. Correctly redirects fd 1 then fd 0.	Prints `hello`. Redirects descriptors via `open` and `dup`.
`ls | grep READ`	Pipe	Prints `README`. Pipes stdout of ls to stdin of grep.	Prints `README`. Implements pipe via kernel `pipe()` system call.
`(echo a; echo b) | wc`	Grouping / Pipe	Prints `2 2 4`. Subshell evaluated and buffered into pipe.	Prints `2 2 4`. Implements nested command tree evaluation.
`echo one; echo two`	Sequence	Prints `one` then `two` sequentially.	Prints `one` then `two` sequentially (`;` operator).
`echo bg &`	Background	Launches async job; prints job PID and outputs `bg`.	Executes asynchronously via child fork without blocking shell.
`ls *`	Wildcard (Glob)	Expands * to all files matching current directory.	Errors: `ls: cannot open *`. Looks for literal filename *.
`echo $HOME`	Variable Expansion	Expands to user home path: `/home/vitalie`.	Prints literal `$HOME`. No environment variable support.
`echo "a b"`	Quote Stripping	Prints `a b`. Quotes stripped by tokenizer.	Prints `"a b"` verbatim with double quotes preserved.
`mkdir d; cd d; ls ..`	PATH search	Creates `d`, enters it, and resolves `ls` via `$PATH`.	`ls ..` fails (`exec ls failed`) due to lack of `PATH` lookup.
`[Arrow Up, Tab, Ctrl+R]`	Line Editing	Recalls history buffer and auto-completes filenames.	Emits raw terminal escape codes; line editor not present.

Table 3.1: Table 3.1 – Comparative analysis of GNU Bash versus the xv6 minimalist shell

3.2 Real Execution Outputs

```
vitalie@Lenovo:~/shtest$ echo hello > f; cat < f
hello
vitalie@Lenovo:~/shtest$ ls | grep READ
README
vitalie@Lenovo:~/shtest$ (echo a; echo b) | wc
      2       2       4
vitalie@Lenovo:~/shtest$ echo one; echo two
one
two
vitalie@Lenovo:~/shtest$ echo bg &
[1] 11330
bg
vitalie@Lenovo:~/shtest$ ls *
args_linux.c  f  README
vitalie@Lenovo:~/shtest$ echo $HOME
/home/vitalie
vitalie@Lenovo:~/shtest$ echo "a␣b"
a b
```

Figure 3.1: Figure 3.1 – Execution output for commands in GNU Bash (Linux)

```
$ echo hello > f; cat < f
hello
$ ls | grep READ
README
$ (echo a; echo b) | wc
2 2 4
$ echo one; echo two
one
two
$ echo bg &
$ bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a␣b"
"a␣b"
```

Figure 3.2: Figure 3.2 – Execution output for identical command stream in xv6 shell

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ ../ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
d              1 22 32
$
```

Figure 3.3: Figure 3.3 – Directory change and binary lookup failure in xv6 due to lack of PATH

3.3 Analysis and Observations

a) **Common features vs. Bash exclusive capabilities:** Both interpreters share fundamental Unix kernel primitives: descriptor redirection (<, >), inter-process anonymous pipes (`|`), command sequencing (`;`), grouping (`()`), and asynchronous execution (`&`). Conversely, features exclusive to Bash are performed entirely in userland: globbing (`*`), variable evaluation (`$`), quote processing, directory searching via `PATH`, command history, and readline tab-completion.

b) **Behavior of `ls` after `cd d` in xv6:** In xv6, `ls ..` fails with `exec ls failed` because xv6 lacks an environment `PATH` mechanism. It attempts to load the binary `ls` strictly from the current working directory `d`. Because the binary exists at the root directory, the invocation succeeds only when addressing it relatively as `../ls ..`.

c) **Impact and ease of feature implementation:** In daily usage, the absence of history and Tab-completion degrades productivity most severely. From an implementation perspective, quote removal would be the easiest feature to add to `user/sh.c`, requiring only token scanning logic to strip matching leading and trailing quote bytes during parsing.

4 PART 3: WHO EXPANDS *? ARGV DIAGNOSTICS

4.1 Implementation of Diagnostic Programs

To establish conclusively whether the operating system kernel or the command interpreter is responsible for wildcard expansion and quote removal, two companion utilities were deployed: `args.c` for xv6 and `args_linux.c` for Linux.

```
// args.c (xv6)
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Figure 4.1: Figure 4.1 – Implementation of user/args.c for parameter verification in xv6

```
// args_linux.c (Linux)
#include <stdio.h>

int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

Figure 4.2: Figure 4.2 – Implementation of `args_linux.c` for parameter verification in Linux

4.2 Experimental Output Comparison

```
vitalie@Lenovo:~/shtest$ ./args one   two      three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

vitalie@Lenovo:~/shtest$ ./args *.c "x␣y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

vitalie@Lenovo:~/shtest$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vitalie]

vitalie@Lenovo:~/shtest$ ./args 'single␣quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 4.3: Figure 4.3 – Parameter parsing output in Linux Bash

```
$ args one   two      three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x␣y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3]␣=␣[y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

Figure 4.4: Figure 4.4 – Parameter parsing output in xv6

4.3 Analysis and Observations

a) **Comparison of `args *.c "x y"`:**

- In Linux Bash: `argc = 3`. The pattern `*.c` was expanded by Bash into matching directory filenames (`args_linux.c`), and the quoted string was treated as a single argument `[x y]` with enclosing quotation marks discarded.

- In xv6: `argc = 4`. The pattern `*.c` was passed literally as `[*.c]`. Due to the absence of quotation logic, the delimiter space divided `"x y"` into two distinct arguments: `["x]` and `[y"]`.

b) **Responsibility for expansion and quote removal:** The **command interpreter (shell)** performs all globbing and quote stripping. The kernel system call `execve` simply receives the finalized pointer array `argv[]`. The application program itself never encounters the original unexpanded wildcard tokens unless expansion is skipped or prevented by escaping.

c) **Handling of consecutive whitespace:** Both interpreters collapse consecutive spaces into a single lexical separator when partitioning command-line words.

5 PART 4: SOURCE CODE ANALYSIS OF USER/SH.C

5.1 Inspecting Command Representations

The implementation of `user/sh.c` was examined using standard text analysis tools:

```
vitalie@Lenovo:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
vitalie@Lenovo:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

```
// user/sh.c lines 8-12
#define EXEC  1
#define REDIR 2
#define PIPE  3
#define LIST  4
#define BACK  5
```

Figure 5.1: Figure 5.1 – Command type definitions in user/sh.c

```
// user/sh.c runcmd() dispatcher
void runcmd(struct cmd *cmd) {
  ...
  switch(cmd->type){
  case EXEC:
    ...
    exec(ecmd->argv[0], ecmd->argv);
    break;
  case REDIR:
    ...
  case LIST:
    ...
  case PIPE:
    ...
  case BACK:
    ...
  }
  exit(0);
}
```

Figure 5.2: Figure 5.2 – The recursive command dispatcher in user/sh.c

```
// user/sh.c line 170 - Handling cd command
if(buf[0] == 'c' && buf[1] == 'd' && buf[2] == ' '){
  // Clumsy but will have to do ...
  buf[strlen(buf)-1] = 0;  // chop \n
  if(chdir(buf+3) < 0)
    fprintf(2, "cannot cd %s\n", buf+3);
  continue;
}
```

Figure 5.3: Figure 5.3 – Built-in handling of the cd command directly in main()

5.2 Analysis and Observations

a) **Command types and triggering characters:**

- `EXEC (1)`: Standard binary execution.
- `REDIR (2)`: Triggered by < (input redirection) or > (output redirection).
- `PIPE (3)`: Triggered by the pipe character `|`.
- `LIST (4)`: Triggered by the sequential semicolon `;`.
- `BACK (5)`: Triggered by the background token &.

b) **Why `cd` cannot be an external executable:** In `user/sh.c`, `cd` is intercepted prior to `fork()` and handled directly via the `chdir()` system call. If `cd` were implemented as an external binary, the shell would fork a child process, execute the utility, change the working directory of the child, and then terminate. The parent shell process would remain unaffected in its original working directory. Therefore, `cd` must intrinsically be a built-in shell operation.

c) **Absence of * and $ parsing:** A comprehensive search across `user/sh.c` demonstrates zero occurrences of wildcard matching or variable parsing logic. The tokenizer simply delimits strings by whitespace and pass-through tokens, explaining why xv6 treats globs and variable symbols as raw literal text.

6 CONCLUSIONS

Through this laboratory work, the architecture of Unix command interpreters was investigated by comparing the educational xv6 shell with GNU Bash under Linux.

The experimental findings clearly established the boundary between operating system kernel facilities and userland shell abstractions. While process isolation, file descriptors, pipes, and process creation are provided directly by kernel system calls (`fork`, `exec`, `pipe`, `dup`), advanced shell ergonomics—such as wildcard expansion (globbing), variable interpolation, quotation interpretation, line editing, and command history—are implemented exclusively within user-space shell code.

Furthermore, diagnosing parameter vector propagation through `args.c` and `args_linux.c` concretely illustrated that the shell formats `argv[]` before the application process starts execution. Finally, source code inspection of `user/sh.c` provided foundational insight into why process directory navigation (`cd`) must be built directly into the parent shell loop, consolidating theoretical knowledge with practical systems programming reality.